

Mini Paper — 2.3 Algorithms

Time: ~35 minutes · Total: 30 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark. Show your traces.

Questions

Q1 [2] (AO1). State what is meant by *Big O notation* and explain why constant factors and lower-order terms are ignored.

Q2 [3] (AO1). State the worst-case time complexity of each of the following, using Big O notation:

(a) Linear search of an unsorted list of n items. (b) Binary search of a sorted list of n items. (c) A naive algorithm whose number of operations doubles each time one extra item is added to the input.

Q3 [5] (AO2). A list contains `[6, 2, 5, 3]`. Carry out an **insertion sort**, showing the state of the list after **each** value has been inserted into its correct position. State the time complexity of insertion sort in the **worst case**.

Q4 [4] (AO2). Trace a **binary search** for the value **19** in the sorted list below (indices 0–6):

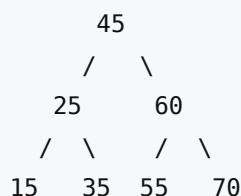
Index:	0	1	2	3	4	5	6
Value:	3	7	11	15	19	23	27

Show the values of `low`, `high` and `mid` at each step, and state how many comparisons of `list[mid]` are needed. State the precondition that must hold for binary search to work.

Q5 [4] (AO1/AO2). A program uses a nested loop in which an outer loop runs n times and, for each outer iteration, an inner loop runs up to n times.

(a) State the time complexity of this algorithm in Big O notation. **[1]** (b) Justify your answer. **[2]** (c) Name one standard sorting algorithm whose average-case complexity is this value. **[1]**

Q6 [4] (AO2). For the binary search tree below, give the **pre-order**, **in-order** and **post-order** traversal output, and state what is special about the in-order output.



Q7 [8] (AO3). Using **Dijkstra's algorithm**, find the shortest distance and the shortest **path** from **A** to **E** in the weighted, undirected graph below. Show your working in a table of tentative distances and the order in which nodes are visited.

Edges (undirected, weighted):

A-B = 4, A-D = 1, B-D = 2, B-C = 3, B-E = 6, C-E = 2, D-E = 7

Mark scheme

Q1 [2] (AO1).

- Big O describes how an algorithm's **time (or space) requirement grows as the input size n increases**, for the worst case (1).
- Constants/lower-order terms are ignored because we care about the **rate of growth / behaviour for large n** , where the dominant term decides scalability (1).

Examiner tip: always tie the definition to "as n increases" — a bare "how fast an algorithm is" scores nothing.

Q2 [3] (AO1).

- (a) **$O(n)$** (1).
- (b) **$O(\log n)$** (1).
- (c) **$O(2^n)$** (1).

Examiner tip: "doubles with each extra item" is the textbook signal for exponential $O(2^n)$ — do not write $O(n^2)$.

Q3 [5] (AO2). Insertion sort on [6, 2, 5, 3]:

i	key	Before	Action	After
1	2	[6, 2, 5, 3]	shift 6 right, insert 2	[2, 6, 5, 3]
2	5	[2, 6, 5, 3]	shift 6 right, insert 5	[2, 5, 6, 3]
3	3	[2, 5, 6, 3]	shift 6,5 right, insert 3	[2, 3, 5, 6] ✓

- Correct result after inserting 2 → [2, 6, 5, 3] (1).
- Correct result after inserting 5 → [2, 5, 6, 3] (1).
- Correct result after inserting 3 → [2, 3, 5, 6] (1).
- Shows shifting of larger elements rather than just swapping / clear working (1).
- Worst-case time complexity **$O(n^2)$** (1).

Examiner tip: insertion sort builds the sorted part at the front — show the list after each insertion, not just the final sorted list.

Q4 [4] (AO2). Binary search for 19 in [3, 7, 11, 15, 19, 23, 27]:

Step	low	high	mid	list[mid]	Comparison	Action
1	0	6	3	15	15 < 19	low = 4
2	4	6	5	23	23 > 19	high = 4
3	4	4	4	19	19 == 19	found at index 4

- Correct mid sequence **3, 5, 4** (1).
- Correct low/high updates (low=4 then high=4) (1).
- Found at **index 4** after **3 comparisons** (1).
- Precondition: the list must be **sorted** (1).

Examiner tip: mid = (low + high) DIV 2 — integer division. Show every low/high update; marks are for the working, not just "found".

Q5 [4] (AO1/AO2).

- (a) **$O(n^2)$** (1).
- (b) The outer loop runs n times and for **each** outer iteration the inner loop runs up to n times, giving up to **$n \times n = n^2$** operations; the n^2 term dominates so the complexity is $O(n^2)$ (2).
- (c) **Bubble sort** OR **insertion sort** (either accepted) (1).

Examiner tip: justify $O(n^2)$ with the multiplication "n passes $\times$ n comparisons" — a nested loop is the classic source of quadratic time.

Q6 [4] (AO2). Traversals of the BST:

- Pre-order (Node, Left, Right): **45, 25, 15, 35, 60, 55, 70** (1).
- In-order (Left, Node, Right): **15, 25, 35, 45, 55, 60, 70** (1).
- Post-order (Left, Right, Node): **15, 35, 25, 55, 70, 60, 45** (1).
- The in-order output is in **ascending sorted order** (1).

Examiner tip: remember the rule by where the Node sits — Pre = node first, In = node middle (sorted), Post = node last. Deduct for any out-of-place node.

Q7 [8] (AO3). Dijkstra's algorithm, A $\rightarrow$ E. Tentative distances from A ("—" = settled/visited):

Step	Current	A	B	C	D	E	Visited
Init	—	0	∞	∞	∞	∞	{}
1	A (0)	—	4	∞	1	∞	{A}
2	D (1)	—	min(4, 1+2)= 3	∞	—	min(∞ , 1+7)=8	{A,D}
3	B (3)	—	—	min(∞ , 3+3)= 6	—	min(8, 3+6)= 9	{A,D,B}
4	C (6)	—	—	—	—	min(9, 6+2)= 8	{A,D,B,C}
5	E (8)	—	—	—	—	—	{A,D,B,C,E}

- Correct initialisation: A = 0, all others = ∞ (1).
- A settled first; D relaxed to 1, B relaxed to 4 (1).
- Visits nodes in correct distance order **A, D, B, C, E** (2).

- Correct relaxation of B to 3 via D, and C to 6 via B (2).
- Shortest distance **A** → **E** = **8** (1).
- Shortest path **A** → **D** → **B** → **C** → **E** ($1 + 2 + 3 + 2 = 8$), found by tracing predecessors $E \leftarrow C \leftarrow B \leftarrow D \leftarrow A$ (1).

Examiner tip: always pick the smallest unvisited tentative distance next, and never re-open a settled node. State both the distance (8) AND the path — many candidates drop the easy path mark.

Total: 30 marks.